

P2079R1

Parallel Executor

Published Proposal, 2020-08-15

This version:

<http://wg21.link/P2079R1>

Authors:

[Ruslan Arutyunyan](#) (Intel)

[Michael Voss](#) (Intel)

Audience:

SG1

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

The authors propose composable Parallel Executor that is suitable for Parallel Algorithms and compute-intense workloads

Table of Contents

1 Motivation

2 Proposed Direction

2.1 Parallel Executor

2.2 Properties

2.2.1 arena_t

2.2.2 wait_context_t

2.3 Parallel executor controls

References

§ 1. Motivation

`static_thread_pool` is a convenient way to create an execution context in place where you need it. However, including it as the sole, standard way to obtain execution resources on the host may promote bad practice that will inadvertently lead to oversubscription and poor composability. In this section, we outline some of the weaknesses of `static_thread_pool` and describe characteristics that an alternative execution context should possess to avoid these weaknesses. We are not arguing for the removal of `static_thread_pool` from [\[P0443R13\]](#) but instead to complement it with at least one additional choice.

The issue with `static_thread_pool` is that it can easily lead to oversubscription. Real-world applications are complicated and, in general, link with many third party libraries. Any shared object (.so) as well as an application itself may create its own `static_thread_pool`. Without alternatives in the standard, this might in fact seem like the only portable choice. However, when there are many `static_thread_pool` instances, the end application will likely, inadvertently request more threads than the number of physical cores available in the hardware, oversubscribing the hardware. For compute-intense workloads, oversubscription often leads to poor performance.

`static_thread_pool` is not suitable for parallel algorithms due to oversubscription and composability issues.

The parallel algorithms (overloads with `ExecutionPolicy`) tend to be extended with an additional overload with `Executor`. But C++17 overloads (without `Executor`) are still there. Since creating separate instances of `static_thread_pool` in each algorithm is not suitable, there needs to be a way to say where those overloads can obtain an appropriate executor for the computation.

§ 2. Proposed Direction

§ 2.1. Parallel Executor

To solve the problems described above we propose to introduce `parallel_executor`. The API is:

```
namespace std::execution {  
    executor auto parallel_executor();  
}
```

- Returns the instance of `parallel_executor`
- Lazy initializes the execution context (thread pool) if it was not initialized previously only when `execute` is called
- The underlying thread pool has the capacity that is equal to `std::thread::hardware_concurrency` by default
- All instances returned from `parallel_executor` function share the same thread pool that is semantically singleton.
- Each `parallel_executor` instance has its own *arena* where work is shared among threads
- Once the thread pool is initialized it remains alive as long as the process exists.

§ 2.2. Properties

This section introduces the properties user can `require`, `prefer` or `query` from `parallel_executor` object.

§ 2.2.1. arena_t

```
namespace std::execution {
    struct arena_t {
        template <class T>
        static constexpr bool is_applicable_property_v = executor<T>;

        static constexpr bool is_requirable = true;
        static constexpr bool is_preferable = true;

        using polymorphic_query_result_type = arena_t;

        constexpr unsigned int concurrency_capacity() const noexcept;

        arena_t() = default;
        arena_t(const arena_t&) = default;
        constexpr arena_t(unsigned int concurrency_capacity);

        constexpr bool operator==(const arena_t&) const;
        constexpr bool operator!=(const arena_t&) const;
    };
}
```

Represents the *arena* of the `parallel_executor` instance where work is shared between threads. Two different `parallel_executor` instances may share the same arena meaning that the work is shared between them. On the other hand, `parallel_executor` instances may be created with different arenas. In that case, the work is not shared between those instances. The work belongs to the arena instance associated with `parallel_executor` the work is executed by.

`concurrency_capacity` controls how many threads at maximum can share the work inside the instance of `parallel_executor`.

Note: There is no guarantee that the number of threads sharing the work is exactly the same as the value of `concurrency_capacity`.

By default `concurrency_capacity` is equal to `std::thread::hardware_concurrency()`

§ 2.2.2. `wait_context_t`

```
namespace std::execution {
    struct wait_context_t {
        template <typename T>
        static constexpr bool is_applicable_property_v = executor<T>;
        static constexpr bool is_requirable = true;
        static constexpr bool is_preferable = true;
        using polymorphic_query_result_type = wait_context_t;

        wait_context_t();
        wait_context_t(const wait_context_t& wc);

        void wait();

        constexpr bool operator==(const wait_context_t&) const;
        constexpr bool operator!=(const wait_context_t&) const;
    };
}
```

Represents the object you can wait on. `wait_context_t` tracks all the work executed and not yet completed by the set of `parallel_executor` instances that share the same `wait_context_t` object and additionally tracks the `parallel_executor` instances in that set with the `std::execution::outstanding_work_t::tracked` property established. One default

constructed `wait_context_t` tracks work independently from another default constructed `wait_context_t`. Two default constructed `wait_context_t` instances are never equal.

§ 2.3. Parallel executor controls

```
namespace std::execution {  
    struct parallel_executor_control {  
        void max_concurrency(unsigned int);  
    }  
}
```

`void max_concurrency(unsigned int)` limits the capacity of `parallel_executor`'s underlying thread pool. This is the upper limit of the *active* threads in the pool. User may have more than one `parallel_executor_control` object at the same time. For the sake of composability, the upper limit is the minimal `max_concurrency` value stored in all constructed but not yet destructed `parallel_executor_control` objects. When one of the objects is destroyed it sets the minimal value of the max concurrency through the remaining not yet destructed `parallel_executor_control` objects.

§ References

§ Informative References

[P0443R13]

Jared Hoberock, Michael Garland, Chris Kohlhoff, Chris Mysisen, Carter Edwards, Gordon Brown, D. S. Hollman, Lee Howes, Kirk Shoop, Lewis Baker, Eric Niebler. [A Unified Executors Proposal for C++](https://wg21.link/p0443r13). 2 March 2020. URL: <https://wg21.link/p0443r13>